

Exemplo Array:

[5, 3, 8, 4, 2]

ordenado:

[2, 3, 4, 5, 8]

1. Selection Sort — Ordenação por Seleção

Ideia principal

O **Selection Sort** procura o menor elemento da parte ainda não ordenada e coloca esse elemento na posição correta.

Imagine:

[5, 3, 8, 4, 2]

Passo 1

Procura o menor elemento:

5 3 8 4 2

↑

menor

O menor é 2.

Troca 2 com o primeiro:

[2, 3, 8, 4, 5]

Agora o 2 está definitivamente na posição correta.

Passo 2

Consideramos apenas:

[3, 8, 4, 5]

O menor é 3.

Ele já está no lugar:

[2, 3, 8, 4, 5]

Passo 3

Na parte:

[8, 4, 5]

o menor é 4.

Troca:

[2, 3, 4, 8, 5]

Passo 4

Menor de:

[8, 5]

é 5.

Resultado:

[2, 3, 4, 5, 8]

Pseudocódigo

para $i = 0$ até $n-2$

 menor = i

 para $j = i+1$ até $n-1$

 se $\text{vetor}[j] < \text{vetor}[\text{menor}]$

 menor = j

 trocar $\text{vetor}[i]$ com $\text{vetor}[\text{menor}]$

Característica

A pergunta que o Selection Sort faz é:

"Qual é o menor elemento que ainda não coloquei no lugar?"

Complexidade:

Melhor caso: $O(n^2)$

Médio caso: $O(n^2)$

Pior caso: $O(n^2)$

2. Insertion Sort — Ordenação por Inserção

A ideia é parecida com **organizar cartas na mão**.

Você pega uma carta e coloca na posição correta entre as que já estão ordenadas.

Começamos:

[5, 3, 8, 4, 2]

Consideramos 5 ordenado:

[5] | [3, 8, 4, 2]

Pegamos 3.

Como $3 < 5$, deslocamos 5:

[3, 5] | [8, 4, 2]

Agora 8:

[3, 5, 8] | [4, 2]

4 precisa entrar entre 3 e 5:

[3, 4, 5, 8] | [2]

Finalmente 2:

[2, 3, 4, 5, 8]

Pseudocódigo

para $i = 1$ até $n-1$

 chave = vetor[i]

$j = i - 1$

 enquanto $j \geq 0$ e $\text{vetor}[j] > \text{chave}$

 vetor[j+1] = vetor[j]

 j--

$\text{vetor}[j+1] = \text{chave}$

A pergunta do Insertion Sort

"Onde este elemento deve ser inserido na parte que já está ordenada?"

Complexidade

<i>Situação</i>	<i>Complexidade</i>
<i>Melhor caso</i>	$O(n)$
<i>Médio caso</i>	$O(n^2)$
<i>Pior caso</i>	$O(n^2)$

Isso acontece porque, se o vetor já estiver praticamente ordenado, ele trabalha muito pouco.

3. Bubble Sort — Ordenação por Bolha

O Bubble Sort compara **elementos vizinhos**.

Se estiverem na ordem errada, troca.

[5, 3, 8, 4, 2]

Primeira comparação:

$5 > 3$

Troca:

[3, 5, 8, 4, 2]

Depois:

$5 < 8$

Não troca.

Depois:

$8 > 4$

Troca:

[3, 5, 4, 8, 2]

Depois:

$8 > 2$

Troca:

[3, 5, 4, 2, 8]

Observe que o maior elemento, 8, "borbulhou" para o final.

Segunda passagem:

[3, 5, 4, 2, 8]

Depois das comparações:

[3, 4, 2, 5, 8]

Terceira:

[3, 2, 4, 5, 8]

Quarta:

[2, 3, 4, 5, 8]

Pseudocódigo

para $i = n-1$ até 1

 para $j = 0$ até $i-1$

 se $\text{vetor}[j] > \text{vetor}[j+1]$

 trocar $\text{vetor}[j]$

 com $\text{vetor}[j+1]$

A pergunta do Bubble Sort

"Esses dois vizinhos estão na ordem correta?"

Complexidade

Melhor caso: $O(n)$ // versão otimizada

Médio caso: $O(n^2)$

Pior caso: $O(n^2)$

4. Merge Sort — Ordenação por Intercalação

Aqui muda bastante a estratégia.

O Merge Sort utiliza:

Dividir para conquistar.

Ele divide o vetor em partes menores até chegar a elementos individuais.

[5, 3, 8, 4, 2]

Divide:

[5, 3] [8, 4, 2]

Divide novamente:

[5] [3] [8] [4, 2]

E:

[4] [2]

Agora começa a **intercalação (merge)**.

[5] + [3]

produz:

[3, 5]

E:

[4] + [2]

produz:

[2, 4]

Depois:

[8] + [2, 4]

produz:

[2, 4, 8]

Finalmente:

[3, 5] + [2, 4, 8]

produz:

[2, 3, 4, 5, 8]

A grande sacada é:

DIVIDIR

↓

[5,3,8,4,2]

↓

[5,3] [8,4,2]

↓

[5][3] [8][4,2]

↓

[5][3] [8][4][2]

↓

INTERCALAR

[3,5] [2,4,8]

↓

[2,3,4,5,8]

Complexidade

Melhor caso: $O(n \log n)$

Médio caso: $O(n \log n)$

Pior caso: $O(n \log n)$

Essa é uma das grandes vantagens do Merge Sort.

A desvantagem é que normalmente precisa de **memória auxiliar** para realizar a intercalação.

5. Quick Sort — Ordenação Rápida

O Quick Sort também utiliza:

Dividir para conquistar.

Mas a estratégia é diferente do Merge Sort.

Ele escolhe um elemento chamado **pivô**.

Por exemplo:

[5, 3, 8, 4, 2]

Vamos escolher:

pivô = 5

Agora reorganizamos os elementos de modo que:

menores que 5 | 5 | maiores que 5

Resultado:

[3, 4, 2] | 5 | [8]

Depois fazemos Quick Sort novamente nas duas partes:

[3, 4, 2]

e:

[8]

Na primeira parte, podemos escolher 3 como pivô:

[2] | 3 | [4]

Resultado final:

[2, 3, 4, 5, 8]

A pergunta do Quick Sort

"Como posso dividir o vetor em elementos menores e maiores que o pivô?"

Complexidade

SITUAÇÃO	COMPLEXIDADE
MELHOR CASO	$O(n \log n)$
MÉDIO CASO	$O(n \log n)$
PIOR CASO	$O(n^2)$

O pior caso pode ocorrer quando o pivô escolhido produz divisões muito desequilibradas.

Por exemplo:

[1, 2, 3, 4, 5, 6, 7]

Se sempre escolhermos o primeiro elemento como pivô:

1 | [2,3,4,5,6,7]

depois:

1 | 2 | [3,4,5,6,7]

e assim por diante.

Comparação geral

Algoritmo	Ideia principal	Melhor	Médio	Pior
Selection	Seleciona o menor	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion	Insere na posição correta	$O(n)$	$O(n^2)$	$O(n^2)$
Bubble	Troca vizinhos	$O(n)^*$	$O(n^2)$	$O(n^2)$
Merge	Divide e intercala	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick	Pivô + particionamento	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

* considerando a versão otimizada, que detecta quando nenhuma troca ocorreu.

Uma forma boa de ensinar para os alunos

Eu resumiria cada algoritmo em **uma frase**:

Selection Sort

"Procuro o menor e coloco na posição."

Insertion Sort

"Pego o elemento e insiro na posição correta."

Bubble Sort

"Comparo vizinhos e faço os maiores borbulharem para o final."

Merge Sort

"Divido, ordeno as partes e depois intercalo."

Quick Sort

"Escolho um pivô, separo menores e maiores e repito."

E uma distinção importante para a aula:

Selection / Insertion / Bubble

↓

$O(n^2)$

↓

algoritmos mais simples

versus:

Merge / Quick

↓

Dividir para conquistar

↓

geralmente $O(n \log n)$

Nota: **visualgo.net** https://algodaily.com/lessons/merge-sort-vs-quick-sort-heap-sort?utm_source=chatgpt.com

Selection Sort

https://youtu.be/ZT_dT8yn48s?list=PL5TJqBvpXQv4l7nH-08fMfyl7aDFNW_fC

-- Bubble Sort

https://youtu.be/GiNPe_678Ms?list=PL5TJqBvpXQv4l7nH-08fMfyl7aDFNW_fC

-- Insertion Sort

https://youtu.be/S5no2qT8_xg?list=PL5TJqBvpXQv4l7nH-08fMfyl7aDFNW_fC

-- Merge Sort

https://youtu.be/5prE6Mz8Vh0?list=PL5TJqBvpXQv4l7nH-08fMfyl7aDFNW_fC

-- Quick Sort

https://youtu.be/wx5juM9bbFo?list=PL5TJqBvpXQv4l7nH-08fMfyl7aDFNW_fC

-- Complexidade

https://youtu.be/v50wSxxlit0?list=PL5TJqBvpXQv4l7nH-08fMfyl7aDFNW_fC

Os termos são:

$$2, \sqrt{\lg n}, \lg n, n, n^{5/4}, n \lg^3 n, n \lg n, 2^n, n^2, 2^{2n}$$

A ordem do maior para o menor, para $n \rightarrow \infty$, é:

$$2^{2n} > 2^n > n^2 > n^{5/4} > n \lg^3 n > n \lg n > n > \lg n > \sqrt{\lg n} > 2$$

